# Group Member Details

Katlego Sentsho u24876489

Patrick Simuyemba u25632354

Enock Nyamweya u25619871

Angelo Anthony u25573552

Ntando Dlamini u25591267

Angela Ramaboea u25445392

Tebogo Mokgwatsane u25042239

# Task 1: Research Findings on Workflow Management Systems

**What a WfMS is.** The Workflow Management Coalition (WfMC) defines workflow as the computerised automation of a business process and a workflow management system (WfMS) as software that defines, manages and executes workflows, with execution order driven by a computer representation of the process logic [1]. All WfMSs share three areas: build-time definition, run-time process control, and run-time interaction with people and applications [1].

**Definition vs instance.** A process definition holds the activities, transition conditions, roles, workflow-relevant data, invoked applications and a version number. An engine interprets it to create and control many independent instances (“cases”) [1][2]. The instance owns control data (the state of each process and activity instance), case data used for routing decisions, and its audit trail [1]. Because processes can run for weeks, engines such as Camunda let running instances finish on the version they started with, send new instances to the latest version, and offer optional migration [5]. A change to a definition therefore never silently changes work already in progress.

**Work items and nesting.** Activities are either atomic or compound (sub-flows) [1][2]. The WfMC describes hierarchical nesting, where a whole process is enacted as one activity of a parent process [1], and BPMN models the same idea with embedded sub-processes and reusable call activities [4]. Control-flow constructs that recur across products are sequence, parallel split and synchronisation, exclusive choice and merge, loops and cancellation; none of the 15 products compared in [2] supported every pattern, so the set of routing constructs must be extensible.

**Assignment, approval and escalation.** Definitions refer to roles rather than named people, and the engine binds roles to participants at run time [1]. The resource patterns catalogue [3] separates push allocation (system chooses) from pull allocation (participant claims), and lists allocation rules such as random, round-robin and shortest-queue, plus delegation and escalation. Approvals are modelled as work items whose outcome feeds routing conditions, and BPMN adds timer, escalation and error events for deadlines and failures [4].

**Events, failures and history.** The WfMC treats the engine as a state machine in which process and activity instances move between states (e.g. initiated, running, suspended, completed, terminated) in response to events and control decisions, with checkpoint and recovery data kept for failure handling [1]. Audit trails, metrics and supervisory operations are exposed through an administration and monitoring interface [1].

**External systems.** Engines do not hard-code applications. They invoke them through an invoked-application interface or application agents, and reach other engines through gateways that map names and data [1].

**Reusability and configurability.** Workflow separates procedure logic from IT support [1], so a generic engine interprets definitions instead of embedding a process. Routing conditions, role binding, allocation rules, escalation rules, invoked applications and versions are all definition data, not engine code [1][3][5].

**Findings that drive the TIVIDY design**

| Recurring responsibility | Expected to vary | Design consequence |
|---|---|---|
| Definition vs instance, versioning | The process blueprint; running cases | Definition built step by step (Builder); each instance owns its state, data and history (State, Memento) |
| Nested, parallel work | Shape and depth of the process | Uniform work-item tree (Composite) |
| Allocation, approval, escalation | Allocation rule; approval levels | Strategy; Chain of Responsibility |
| Events, audit, undo, extra behaviour | Who reacts; what is reversible | Observer; Command; Memento; Decorator |

| Recurring responsibility | Expected to vary | Design consequence |
|---|---|---|
| External systems, participants | Foreign APIs; who talks to whom | Adapter; Mediator; single entry point (Facade) |

### References


**[1]** Hollingsworth, D. (1995). *The Workflow Reference Model* (WfMC Document TC00-1003, Issue 1.1). Workflow Management Coalition. https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/refmodel/rmv1-16.html

**[2]** van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B., & Barros, A.P. (2003). Workflow Patterns. *Distributed and Parallel Databases*, 14(1), 5–51. https://doi.org/10.1023/A:1022883727209 (open copy: pure.tue.nl)

**[3]** Russell, N., van der Aalst, W.M.P., ter Hofstede, A.H.M., & Edmond, D. (2005). Workflow Resource Patterns: Identification, Representation and Tool Support. In *CAiSE 2005*, LNCS 3520, pp. 216–232. Springer. https://doi.org/10.1007/11431855_16

**[4]** Object Management Group. (2011). *Business Process Model and Notation (BPMN), Version 2.0* (formal/2011-01-03). https://www.omg.org/spec/BPMN/2.0

**[5]** Camunda. (n.d.). *Versioning process definitions* (Best practices). Accessed 3 October 2026. https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/

**[6]** Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. ISBN 978-0-201-63361-0.

# Task 2: Define the Scenario and System

## 2.1 Selected Industries and Workflow Processes

TIVIDY is a generic workflow management system designed to define, execute and monitor structured processes across different industries. To demonstrate the reusability and flexibility of the system, two workflows from distinct industries were selected: **Education** and **Logistics**.

The selected processes are University Admissions and Bursary Processing, representing the education industry, and Shipment Exception Management, representing the logistics industry.

Although these workflows involve different participants, activities, decisions and business rules, both are executed using the same underlying TIVIDY workflow engine. The industry-specific differences are represented through workflow definitions, configuration, participants, roles and applicable integrations, without requiring changes to the generic engine.

**Industry 1: Education**

- **Process:** University Admissions and Bursary Processing
- **Industry:** Education / Higher Education
- **Purpose:** To coordinate the processing of student applications, academic eligibility checks, bursary assessments, approval decisions and final admission outcomes.
- **Participants:** Applicants, Admissions Officers, Academic Reviewers, Financial Aid Officers, Bursary Committee members and relevant administrative personnel.
- **Input Data:** Applicant details, programme selection, identification information, academic results, supporting documents and financial information.
- **Main Activities:** Application submission, document verification, academic assessment, eligibility decisions, bursary evaluation, approval or rejection, and applicant notification.
- **Decision and Exception Paths:** Incomplete or invalid documents, unsuccessful academic assessment, bursary rejection, approval escalation and unsuccessful external verification.
- **Expected Outcome:** An application is processed to an appropriate final outcome, with its activities, decisions and history recorded by TIVIDY.

**Industry 2: Logistics**

- **Process:** Shipment Exception Management
- **Industry:** Logistics and Supply Chain Management
- **Purpose:** To manage operational exceptions affecting shipments through structured investigation, task assignment, corrective actions, decisions and resolution tracking.
- **Participants:** Logistics Coordinators, Warehouse Personnel, Dispatch Officers, Transport or Delivery Personnel, Supervisors and other authorised operations staff.
- **Input Data:** Shipment identification, shipment status, exception category, supporting shipment information and relevant operational details.
- **Main Activities:** Exception registration, initial validation, assignment to responsible personnel, investigation, corrective action, escalation or approval where required, and final resolution.
- **Decision and Exception Paths:** Missing or invalid shipment information, failed validation, unresolved shipment exceptions, rejected corrective actions and escalation to an authorised participant.
- **Expected Outcome:** The shipment exception is resolved or appropriately closed, with an auditable history of actions, decisions, assignments and status changes.

## 2.2 Demonstrating the Reusability of TIVIDY

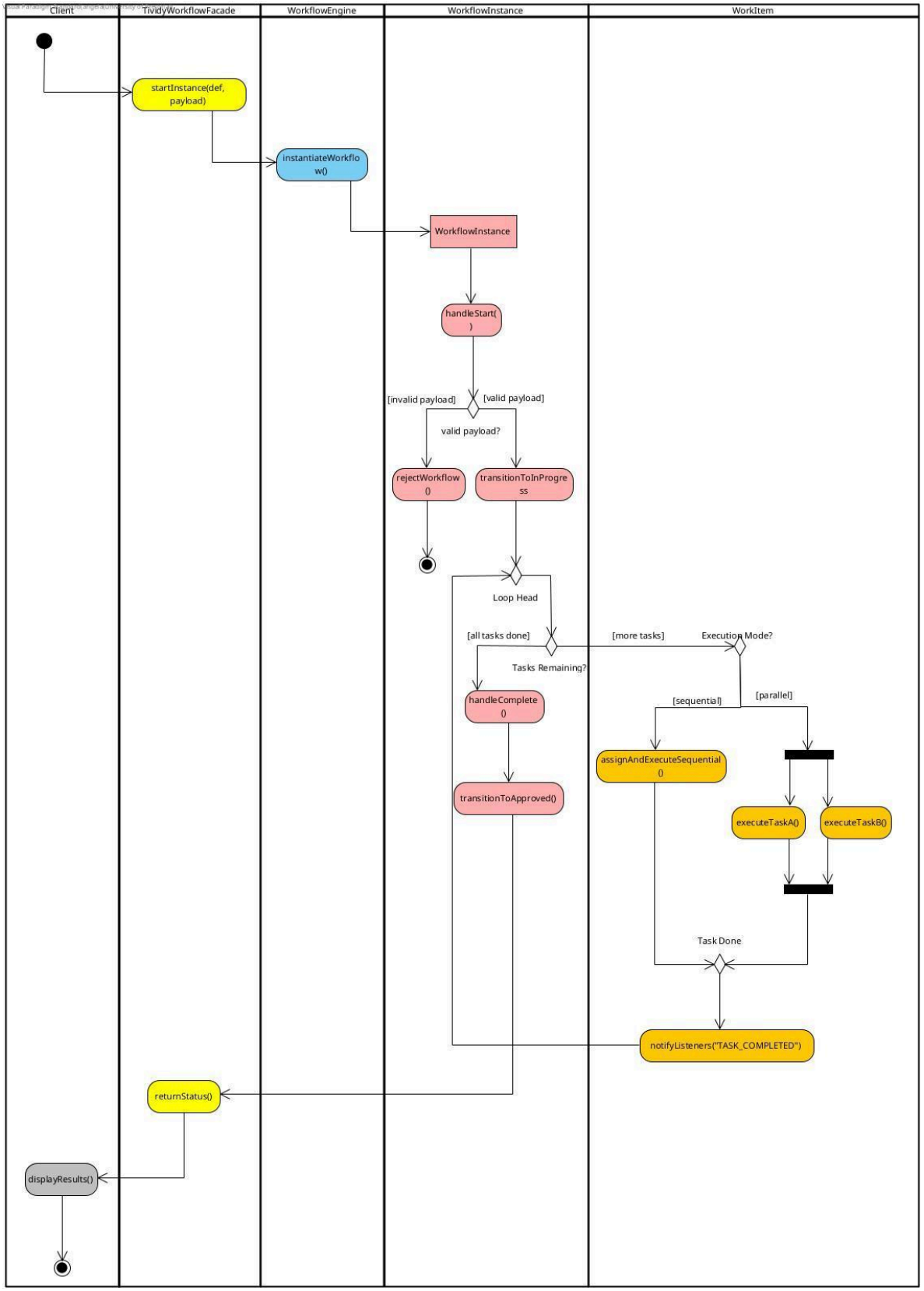
The two scenarios demonstrate that TIVIDY is not restricted to a specific industry or business process.

The Education workflow focuses on processing student applications and bursary decisions, while the Logistics workflow focuses on investigating and resolving shipment exceptions. Both processes require work-item management, participants, routing, dependencies, decision-making, state transitions and execution history.

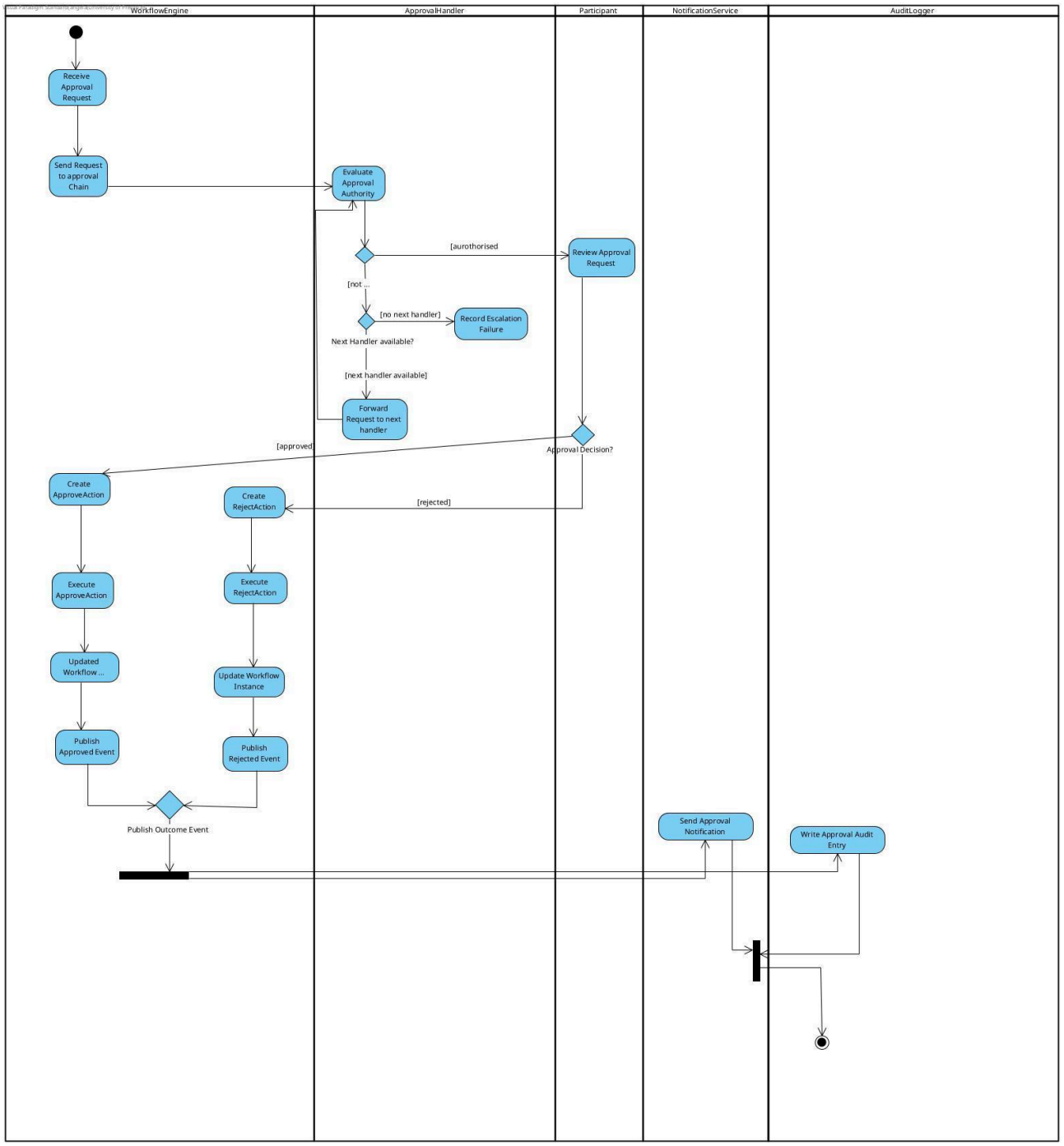
TIVIDY addresses these shared requirements through its generic workflow engine. Each industry supplies its own workflow definition, business data and configuration, while the core engine classes remain unchanged.

This separation demonstrates the central objective of the project: **one reusable workflow engine capable of executing multiple industry-specific workflows without duplicating the underlying workflow-management logic.**

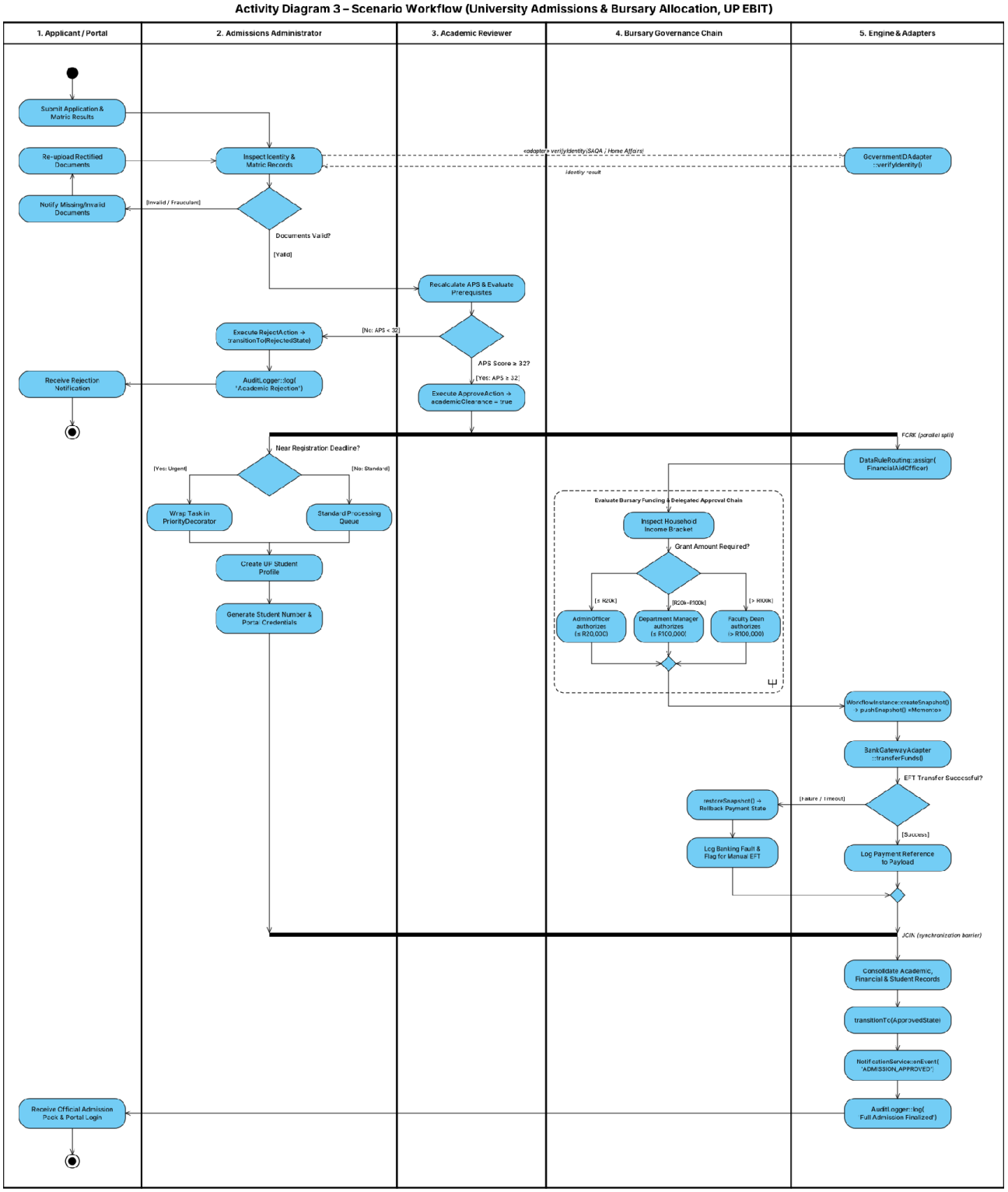
# Task 3: UML Activity Diagrams




*Activity Diagram 1: Generic workflow execution, from instance creation to completion*




*Activity Diagram 2: Approval and escalation*




*Activity Diagram 3: Scenario workflow (University Admissions and Bursary Processing)*

# Task 4: Design Patterns

TIVIDY is designed as a generic workflow manager, so each pattern below is justified by a problem of the engine itself, drawing on the research in Task 1. None of the participant classes refer to a specific process. The University Admissions and Bursary Processing scenario appears only in the last row of each table, as one example of how a process configures the engine; removing those rows leaves a complete design. Thirteen patterns are proposed (the minimum is ten). Reference numbers refer to the list under Task 1; [6] is the GoF catalogue.

| # | Pattern | Category | Workflow problem it solves |
|---|---|---|---|
| 1 | **Builder** | Creational | Definition (blueprint) must be assembled step by step and kept separate from running instances |
| 2 | **Composite** | Structural | Work is nested: tasks and sub-processes must be treated uniformly |
| 3 | **State** | Behavioural | Operations valid in one work-item lifecycle state are invalid in another |
| 4 | **Strategy** | Behavioural | Organisations allocate work by different rules (role, workload, data) |
| 5 | **Chain of Responsibility** | Behavioural | Approval may need one or several levels, with escalation by rule or deadline |
| 6 | **Command** | Behavioural | User and system actions on work items must be executed uniformly, logged and undone |
| 7 | **Memento** | Behavioural | An instance must roll back to a checkpoint after a partial failure |
| 8 | **Observer** | Behavioural | Several independent parts must react to the same workflow event |
| 9 | **Decorator** | Structural | A work item gains extra behaviour without becoming a new kind of item |
| 10 | **Adapter** | Structural | External systems expose interfaces not designed for TIVIDY |
| 11 | **Mediator** | Behavioural | Work items must trigger each other without referencing each other |
| 12 | **Facade** | Structural | Clients need one simple entry point to many engine subsystems |
| 13 | **Singleton** | Creational | Centralised event bus providing a single source of truth for audit and telemetry, without passing a dispatcher through every constructor in deep task trees |

## 4.1 Builder (Creational)

| | |
|---|---|
| **Design problem** | A workflow definition contains a work-item tree, routing rules, dependencies, approval chains and a version number [1]. It is complex to assemble, must be validated before use, and must stay separate from the instances created from it, so that many independent instances can share one blueprint [1][5]. |
| **GoF participants → TIVIDY classes** | **Director:** WorkflowDirector<br>**Builder:** WorkflowDefinitionBuilder (addTask(), addSubProcess(), setRouting(), addDependency(), setApprovalChain(), getResult())<br>**ConcreteBuilder:** FileWorkflowDefinitionBuilder<br>**Product:** WorkflowDefinition (versioned) |
| **Collaboration** | WorkflowDirector calls the construction steps in order through the abstract WorkflowDefinitionBuilder. The concrete builder creates the work-item tree, attaches a RoutingStrategy, approval chain, decorators and dependency rules where required, and returns a versioned WorkflowDefinition.<br>WorkflowEngine later creates WorkflowInstance objects from that definition. Each instance receives its own copy of the tree, its own state, data and history, so a change to one instance never changes the definition or any other instance. |
| **What varies** | Which process is being defined. A new process is a new file that is read by the builder who creates the definition based on the specifications in the file. All the participating classes remain unchanged. New concrete builders that builders the definition based on another type of payload can easily be added in the future. |

| | |
|---|---|
| **Why this pattern** | It separates how a definition is assembled from what it represents, and keeps every process-specific detail out of the engine. Building step by step also keeps the blueprint separate from running instances, which the research identified as central [1][5]. |
| **Example in the scenario** | A txt file for the university applications workflow is supplied to the builder, the builder reads the files and builds the structure, adds routing, dependencies, decorators and much on how the workflow. |

## 4.2 Composite (Structural)

| | |
|---|---|
| **Design problem** | Real processes contain stages, individual tasks and nested sub-processes, some sequential and some parallel [1][2][4]. The engine must start, track, cancel and report on a single task or a whole branch in the same way, without knowing how children are stored. |
| **GoF participants → TIVIDY classes** | **Component:** WorkItem<br>**Leaf:** Task<br>**Composite:** SubProcess (owns child WorkItems; execution mode sequential or parallel) |
| **Collaboration** | WorkflowInstance owns one root SubProcess. Operations such as start(), cancel() and isComplete() can be called on any WorkItem; a SubProcess forwards them to its children according to its execution mode and completes when its required children complete. Children are owned by composition and destroyed with their parent. |
| **What varies** | The shape and depth of the process. New kinds of work are new WorkItem subclasses with no change to the engine. |
| **Why this pattern** | Hierarchical sub-processes are standard in the WfMC model and in BPMN [1][4]. Composite gives work and groups of work one interface and removes any “is this a sub-process?” type checks from the engine. |
| **Example in the scenario** | The bursary branch is a SubProcess containing the income check, approval and payment tasks. |

## 4.3 State (Behavioural)

| | |
|---|---|
| **Design problem** | Every work item moves through a lifecycle: created, made available, assigned, started, completed, rejected, cancelled or escalated. Operations valid in one state are invalid in another; for example a completed item cannot be started again. Encoding this as a status flag with if/else chains is forbidden by the project rules, and the WfMC describes activity instances as state machines [1]. |
| **GoF participants → TIVIDY classes** | **Context:** WorkItem<br>**State:** WorkItemState<br>**ConcreteStates:** CreatedState, AvailableState, AssignedState, InProgressState, CompletedState, RejectedState, CancelledState, EscalatedState |
| **Collaboration** | WorkItem delegates every lifecycle request (makeAvailable(), assign(), start(), complete(), reject(), cancel(), escalate()) to its current state object. The state either performs the transition and sets the next state, or refuses the request. WorkflowInstance keeps a simple overall status (running, suspended, completed or terminated) derived from its root work item. |
| **What varies** | The lifecycle rules: which operations are legal in which state and what each transition triggers. A new state is a new class. |
| **Why this pattern** | Each state’s rules live in one class instead of in conditionals. Approval outcomes such as approved or rejected are results of an approval task, not states of the whole workflow, so the same lifecycle applies to any process. The transitions match the State Diagram (Task 6). |
| **Example in the scenario** | The academic review task moves from Available to Assigned to InProgress, and then to Completed with the outcome “approved”, or to Rejected. |

## 4.4 Strategy (Behavioural)

| | |
|---|---|
| **Design problem** | Organisations allocate work differently: by role, by workload, or by a rule over instance data [3]. Different tasks in the same process may also need different rules. The task classes must not change when the rule changes. |
| **GoF participants → TIVIDY classes** | **Context:** Task<br>**Strategy:** RoutingStrategy (assign(task, participants))<br>**ConcreteStrategies:** RoleBasedRouting, WorkloadRouting, DataRuleRouting (all part of the generic engine) |
| **Collaboration** | When a Task becomes available it asks its RoutingStrategy for a Participant. RoleBasedRouting picks an available participant holding a configured Role; WorkloadRouting picks the eligible participant with the shortest queue; DataRuleRouting reads a configured field from the instance data and maps its value to a role. |
| **What varies** | The allocation algorithm, chosen per task in the workflow definition. |
| **Why this pattern** | The resource-patterns catalogue treats allocation rules (role-based, round-robin, shortest queue and others) as interchangeable choices [3]. Strategy keeps them out of Task and avoids a switch on allocation type. The strategies are generic; a process only supplies their settings. |
| **Example in the scenario** | Bursary tasks use DataRuleRouting configured on the household-income field; review tasks use RoleBasedRouting. |

## 4.5 Chain of Responsibility (Behavioural)

| | |
|---|---|
| **Design problem** | Approval may involve one person or several levels, and a request must escalate when it is beyond an approver's authority or when nobody responds before a deadline [3][4]. The task requesting approval must not know who will decide. |
| **GoF participants → TIVIDY classes** | **Handler:** ApprovalHandler (handle(request), nextHandler)<br>**ConcreteHandler:** ConcreteApprovalHandler, configured with an approver Role, an ApprovalRule (canApprove(request)) and a deadline |
| **Collaboration** | An approval task passes an ApprovalRequest to the first handler. If the handler's rule accepts the request, its approver decides; otherwise, or when the deadline passes, the request is forwarded to nextHandler. The decision returns to the task as its outcome, and the workflow continues or rejects. |
| **What varies** | The number of levels, their order, their rules and their deadlines. Changing the hierarchy means configuring handlers, not editing the engine. |
| **Why this pattern** | Hierarchical escalation is naturally a chain. Because the acceptance test is a rule object rather than a fixed amount, the same handler class serves amount limits, risk levels or any other criterion. |
| **Example in the scenario** | Bursary approval uses amount-based rules: a financial aid officer, then a manager, then the dean. |

## 4.6 Command (Behavioural)

| | |
|---|---|
| **Design problem** | Users and the system act on work items: claiming, completing, rejecting, delegating or cancelling them. These actions must be executed uniformly, recorded for audit and undone where sensible [1], and must not be coded directly into a client or a task. |
| **GoF participants → TIVIDY classes** | **Invoker:** WorkflowEngine (executeAction())<br>**Command:** WorkItemAction (execute(), undo())<br>**ConcreteCommands:** ClaimAction, CompleteAction (carries an outcome), RejectAction, DelegateAction, CancelAction<br>**Receiver:** WorkItem<br>**Client:** TividyWorkflowFacade (creates the action for a user's request) |

| | |
|---|---|
| **Collaboration** | The facade turns a client request into an action and passes it to WorkflowEngine.executeAction(). The action calls the matching operation on the WorkItem, which delegates to its current WorkItemState (pattern 3). The engine stores executed actions as history and can call undo(). |
| **What varies** | Which actions exist and how they are queued, logged or undone. |
| **Why this pattern** | Requests become objects, so the engine depends only on WorkItemAction and not on a switch over decision types. Approving is simply a CompleteAction with the outcome “approved”, so no process-specific command is needed. A new kind of action is a new subclass. |
| **Example in the scenario** | An academic reviewer’s decision is a CompleteAction on the review task. |

## 4.7 Memento (Behavioural)

| | |
|---|---|
| **Design problem** | Before a risky step, especially a call to an external system, a running instance needs a recovery point. If the step fails, the instance must return to its earlier state without exposing its internals [1]. |
| **GoF participants → TIVIDY classes** | **Originator:** WorkflowInstance (creates and restores snapshots)<br>**Memento:** WorkflowSnapshot (state and data; opaque to others)<br>**Caretaker:** HistoryLog |
| **Collaboration** | Before any work item marked as a checkpoint runs, WorkflowInstance creates a WorkflowSnapshot and HistoryLog stores it. If the step fails, HistoryLog supplies the snapshot and the instance calls restoreSnapshot(). HistoryLog never reads inside the snapshot. |
| **What varies** | What is saved, which steps are checkpoints and how many snapshots are kept. |
| **Why this pattern** | Rollback needs the full earlier state while the instance keeps its data encapsulated. The stored snapshots also give the system a history it can use to explain what happened. |
| **Example in the scenario** | A checkpoint is taken before the bank payment task. |

## 4.8 Observer (Behavioural)

| | |
|---|---|
| **Design problem** | When something happens in a workflow (an item completes, fails or escalates), several independent parts must react, such as notification, auditing and monitoring. Work items must not know how these parts work [1]. |
| **GoF participants → TIVIDY classes** | **Subject: EventDispatcher (the Singleton of 4.13); each WorkItem publishes its events through it**<br>**Observer:** WorkflowEventListener (onEvent(WorkflowEvent))<br>**ConcreteObservers:** NotificationService, AuditLogger<br>**Event:** WorkflowEvent with a generic type (ITEM_COMPLETED, ITEM_FAILED, ITEM_ESCALATED and so on), the item id and the instance id |
| **Collaboration** | Listeners subscribe once to EventDispatcher. On each lifecycle transition the WorkItem publishes a WorkflowEvent through EventDispatcher, which forwards it to all of them: NotificationService informs the relevant participant and AuditLogger records the event. The same mechanism produces the final outcome notification when the workflow completes. |
| **What varies** | The set of interested parties. A new reaction is a new listener with no change to WorkItem. |
| **Why this pattern** | One event needs one-to-many notification. Direct calls would couple every work item to every reaction, and generic event types keep scenario vocabulary out of the engine. |
| **Example in the scenario** | When the offer task completes, the applicant is notified and the audit record is written. |

## 4.9 Decorator (Structural)

| | |
|---|---|
| **Design problem** | An individual work item may need extra behaviour such as priority handling, extra auditing, validation or access checks, without becoming a different kind of work item. |
| **GoF participants → TIVIDY classes** | **Component:** WorkItem<br>**Decorator:** WorkItemDecorator (wraps a WorkItem)<br>**ConcreteDecorators:** PriorityDecorator, AuditDecorator |
| **Collaboration** | A decorator wraps any WorkItem, delegates to it and adds its own behaviour before or after the call. Because decorators share the WorkItem interface, they can wrap any Task or SubProcess in the tree and can be stacked. |
| **What varies** | Which extra behaviours apply and in what combination, chosen per item in the definition or at run time. |
| **Why this pattern** | Subclasses for every combination (such as an urgent, audited task) would multiply classes. Decorator adds behaviour per object and leaves the base task unchanged. |
| **Example in the scenario** | The payment task is wrapped in an AuditDecorator, and late applications receive a PriorityDecorator. |

## 4.10 Adapter (Structural)

| | |
|---|---|
| **Design problem** | Workflows must call external applications whose interfaces were not designed for TIVIDY. The WfMC model keeps the engine independent of specific applications through a generic invoked-application interface [1]. |
| **GoF participants → TIVIDY classes** | **Target:** ExternalService (invoke(ServiceRequest): ServiceResult)<br>**Adapter:** one adapter per external system, supplied as a plug-in<br>**Adaptee:** the external system's own API<br>**Client:** Task configured with an ExternalService |
| **Collaboration** | A task calls invoke() on its configured ExternalService. The adapter translates the request into the external system's API and converts the result back; the task stores the result in the instance data. A failure triggers a snapshot restore (pattern 7). |
| **What varies** | The external systems and their APIs. A new integration means writing a new adapter only. |
| **Why this pattern** | The external interfaces cannot be changed, and the engine should depend on one generic service interface. Adapter supplies that translation layer and keeps every integration outside the engine. |
| **Example in the scenario** | GovernmentIDAdapter (identity verification) and BankGatewayAdapter (bursary payment). |

## 4.11 Mediator (Behavioural)

| | |
|---|---|
| **Design problem** | Completing one work item can make others available, and items in parallel branches may depend on one another [2]. If items referenced each other directly, every change to the process would change the items themselves. |
| **GoF participants → TIVIDY classes** | **Mediator:** WorkflowMediator<br>**ConcreteMediator:** DependencyMediator (reads the dependency rules stored in the WorkflowDefinition)<br>**Colleague:** WorkItem |
| **Collaboration** | A work item notifies the mediator when its state changes. The mediator checks the definition's dependency rules and makes the next items available, or cancels items that are no longer needed. Work items never call one another. |
| **What varies** | The dependency rules, which are definition data rather than code. |
| **Why this pattern** | Coordination rules live in one place, and because they are data, a new process needs no new mediator class. Unlike Observer, which broadcasts to listeners that do not talk back, the mediator acts back on its colleagues. |

| | |
|---|---|
| **Example in the scenario** | Completing academic clearance makes the enrolment and bursary branches available. |

## 4.12 Facade (Structural)

| | |
|---|---|
| **Design problem** | TIVIDY has many collaborating subsystems (builders, engine, instances, history and services). A client such as a portal or an administration console should not construct workflow objects itself or know the order of calls. |
| **GoF participants → TIVIDY classes** | **Facade:** TividyWorkflowFacade (registerDefinition(), startInstance(), performAction(), getStatus())<br>**Subsystem classes:** WorkflowEngine, WorkflowDirector, HistoryLog |
| **Collaboration** | The client calls one facade operation, for example startInstance(definition, data). The facade forwards the request to the right subsystems in the right order; WorkflowEngine creates the WorkflowInstance and starts it. The client never touches the internal classes. |
| **What varies** | The subsystems behind the facade. Only the small facade interface needs to stay stable. |
| **Why this pattern** | The system has many independent responsibilities; Facade gives them a single entry point and keeps clients decoupled from internal structure. |
| **Example in the scenario** | The applicant portal calls startInstance() when an application is submitted. |

## 4.13 Singleton (Creational)

| | |
|---|---|
| **Design problem** | In a deep composite workflow hierarchy, lifecycle events must be broadcast system-wide. Passing an event dispatcher pointer down through every stage, task and state constructor pollutes constructors and couples components. Having several dispatchers would also fragment the audit trail and give inconsistent telemetry. |
| **GoF participants → TIVIDY classes** | **Singleton: EventDispatcher (also the Subject of Observer, 4.8)**<br>**getInstance():** static EventDispatcher& EventDispatcher::getInstance()<br>**Sole instance:** static local EventDispatcher instance inside getInstance() (Meyers' Singleton) |
| **Collaboration** | At start-up the engine registers the observers (NotificationService, AuditLogger, MonitoringService) once through EventDispatcher::getInstance().attachListener(...). During execution, any WorkItem or state anywhere in the tree calls EventDispatcher::getInstance().notifyListeners(event) to publish a transition. A private constructor and deleted copy constructor and assignment operator prevent a second instance. |
| **What varies** | The set of subscribed observers. The global access point and the dispatch mechanism stay the same. |
| **Why this pattern** | It guarantees a single point of truth for every workflow transition, avoids passing a dispatcher through every constructor, and adds a second Creational pattern alongside Builder. A C++11 Meyers' Singleton gives lazy, thread-safe initialisation with no heap allocation to leak. |
| **Example in the scenario** | When the “APS Evaluation” task completes, it calls EventDispatcher::getInstance().notifyListeners(COMPLETED) directly, without a dispatcher pointer being passed in from AdmissionsWorkflowBuilder. |

## 4.14 How the patterns form one system

The patterns form one engine. At design time a concrete builder, driven by WorkflowDirector (Builder), produces a versioned WorkflowDefinition whose WorkItem tree (Composite) carries a RoutingStrategy per task (Strategy), approval chains (Chain of Responsibility), dependency rules, decorators (Decorator) and external services (Adapter). At run time a client calls TividyWorkflowFacade (Facade), and WorkflowEngine creates a WorkflowInstance with its own copy of the tree. Each work item follows its lifecycle (State); users act on items through actions executed by the engine (Command); every transition publishes a WorkflowEvent through EventDispatcher (Singleton) to listeners such as NotificationService and AuditLogger (Observer) and informs DependencyMediator, which makes the next items available (Mediator). Before checkpoint steps the instance

saves a WorkflowSnapshot in HistoryLog (Memento). A class may take part in several collaborations: WorkItem is the Component of Composite and Decorator, the Context of State, the publisher of Observer events, the Receiver of Command and a Colleague of Mediator, EventDispatcher is the Subject of Observer, and WorkflowInstance is the Originator of Memento. A new process, such as an insurance claim, therefore needs only a new concrete builder, its configuration and adapters for its external systems. The EventDispatcher Singleton serves as the central circulatory system, allowing tasks and states across all subsystems to trigger notifications, audit logging and metrics without tight coupling. These roles are shown with colours and a legend in the UML Class Diagram (Task 5).

## 4.15 Design Evolution and Justification of Architectural Refinements

The initial TIVIDY architecture developed during Practical 6 was designed as a generic workflow management system capable of supporting different business processes. However, during the review and refinement of the design, several limitations were identified that could affect its reusability, flexibility and maintainability.

Although the core architecture was generic in principle, parts of the design were still influenced by the original University Admissions and Bursary Processing scenario. Some workflow behaviour was described in terms of specific admissions activities, approval processes and external integrations. This created a risk that the system would become too closely aligned with the requirements of one industry rather than providing a completely reusable workflow engine.

For example, the original scenario described processes such as academic evaluation, bursary approval and payment processing, together with integrations such as GovernmentIDAdapter and BankGatewayAdapter. These components were appropriate for demonstrating the education workflow, but they could not represent the different operational requirements of a logistics workflow on their own.

The problem was not the existence of industry-specific components, since such components are necessary. Rather, the concern was ensuring that these components remained outside the generic engine and that the engine did not depend on their specific business rules, data fields or outcomes.

A second concern was the **overlap in responsibilities between certain design patterns**. While the patterns individually addressed valid workflow-management problems, their interactions were not always clearly separated in the initial design. This introduced the possibility of duplicated logic, increased coupling and difficulties determining which component should control a particular operation.

The following examples motivated the architectural refinements.

### 1. Composite and Mediator — Overlapping Execution Responsibilities

In the initial design, the Composite pattern represented tasks and subprocesses, with SubProcess also described as supporting sequential and parallel execution modes.

However, the Mediator pattern was responsible for evaluating dependencies and determining when subsequent work items could become available.

This created a potential overlap, because both SubProcess and DependencyMediator could appear responsible for determining the order in which activities execute.

For example, in the Admissions workflow, document verification must be completed before academic assessment can begin. In the Logistics workflow, an exception must be registered before investigation can begin.

In both cases, the ordering of activities should be determined by workflow dependency rules rather than by execution logic embedded within the subprocess structure.

**Refinement:** The responsibilities were separated so that Composite manages the hierarchical structure of work, while Mediator coordinates workflow progression, including sequence, branching and synchronisation.

This allows different workflow structures and execution paths to be configured without modifying the generic SubProcess class.

## 2. State and Command — Overlapping Action and Transition Responsibilities

The State and Command patterns both participate in workflow operations, but their responsibilities needed to be clearly distinguished.

The Command pattern represents actions requested by users or the system, while the State pattern determines whether those actions are permitted in the current lifecycle state.

Without a clear separation, action classes could begin containing lifecycle validation logic that should belong to the state classes.

For example, when an Admissions Officer completes a document-verification task, the command should represent the completion request. It should not independently determine whether the task is currently eligible for completion.

Similarly, a Logistics Officer should not be able to complete an investigation task that has not been started.

**Refinement:** Command classes encapsulate and execute requested operations, while WorkItemState and its concrete states enforce valid lifecycle transitions.

This prevents duplication of transition rules and ensures consistent behaviour across all workflow scenarios.

## 3. Observer and Mediator — Overlapping Event and Coordination Responsibilities

Both Observer and Mediator respond to changes occurring within the workflow. However, their purposes are different.

Observer is responsible for notifying interested components about events, while Mediator is responsible for coordinating dependencies between work items.

Without this distinction, event listeners could become responsible for activating subsequent tasks, or the mediator could become responsible for external notifications and logging.

For example, when academic assessment is completed, the Admissions workflow may need to activate the next stage and record the event in the audit history.

Similarly, when a shipment investigation is completed, the Logistics workflow may need to activate a corrective-action task and notify the relevant monitoring services.

**Refinement:** DependencyMediator is responsible for workflow progression, while EventDispatcher distributes events to registered listeners such as AuditLogger, NotificationService and MonitoringService.

This ensures that workflow coordination and event reporting remain separate responsibilities.

## 4. Strategy and Chain of Responsibility — Overlapping Assignment and Approval Responsibilities

The original design included both routing strategies and approval chains to manage the assignment of work to participants.

These patterns are closely related because both can influence which participant receives a task.

For example, the Admissions workflow may assign a bursary assessment to a Financial Aid Officer before escalating an approval request to a manager.

In the Logistics workflow, an exception may initially be assigned to an operations employee before being escalated to a supervisor.

Without clear responsibilities, the routing strategy and approval chain could both attempt to determine the appropriate participant.

**Refinement:** Strategy determines the method used to allocate work, while Chain of Responsibility evaluates approval or escalation requirements through a sequence of handlers. Where appropriate, an approval-routing strategy uses the chain to identify an authorised participant.

This makes ordinary task assignment and authority-based escalation independently configurable without introducing separate routing logic into each industry.

### 5. Industry-Specific Behaviour and Generic Workflow Execution

Another important refinement involved ensuring that industry-specific processes were represented through configuration and plug-in components rather than hard-coded workflow behaviour.

The initial design was demonstrated mainly through the Admissions and Bursary Processing scenario. Consequently, the examples and proposed integrations focused heavily on academic requirements, bursary decisions and student administration.

The introduction of Logistics Shipment Exception Management highlighted the importance of separating the generic workflow engine from the details of any particular business process.

For example, the engine should not need to understand what an Admission Point Score means, whether a student qualifies for a bursary, or why a shipment has been delayed.

Instead, it should process generic work items, evaluate configured dependency conditions, assign participants, manage state transitions and record execution history.

**Refinement:** Business-specific activities, role names, data fields, approval thresholds, dependency rules and external adapters are treated as scenario configuration or plug-in components.

This enables the Education and Logistics workflows to operate through the same underlying engine without requiring separate workflow-management implementations.

## 4.16 Overall Justification of the Redesign

The architectural refinements were undertaken to address two key weaknesses identified during the review of the initial design: the potential for scenario-specific behaviour to influence the generic engine, and the overlap of responsibilities between collaborating design patterns.

The revised architecture improves separation of concerns by assigning each pattern a clearly defined responsibility. Composite represents work structure, Mediator manages dependencies, State enforces lifecycle rules, Command encapsulates actions, Observer distributes events, Strategy handles work allocation, and Chain of Responsibility manages approval or escalation decisions.

At the same time, the separation between the generic engine and industry-specific configuration was strengthened. This allows different workflows to be introduced without modifying the fundamental classes responsible for workflow execution.

The addition of the Logistics scenario alongside University Admissions and Bursary Processing provides an opportunity to demonstrate this reusability across two distinct industries.

**Ultimately, the purpose of the redesign was not to introduce more design patterns, but to ensure that the existing patterns collaborate coherently, avoid duplicated responsibilities and support a truly reusable workflow management system.**

# Task 5: UML Class Diagram



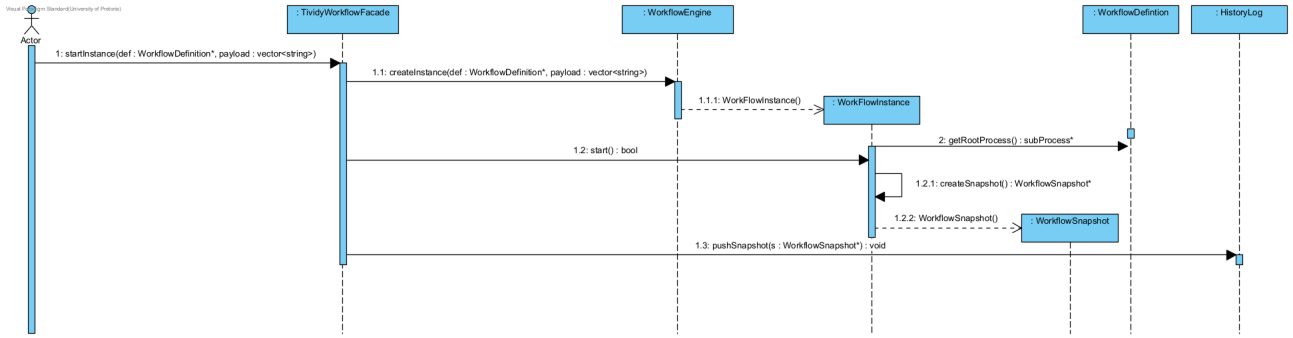
*UML Class Diagram: colours mark the pattern collaborations (see legend)*



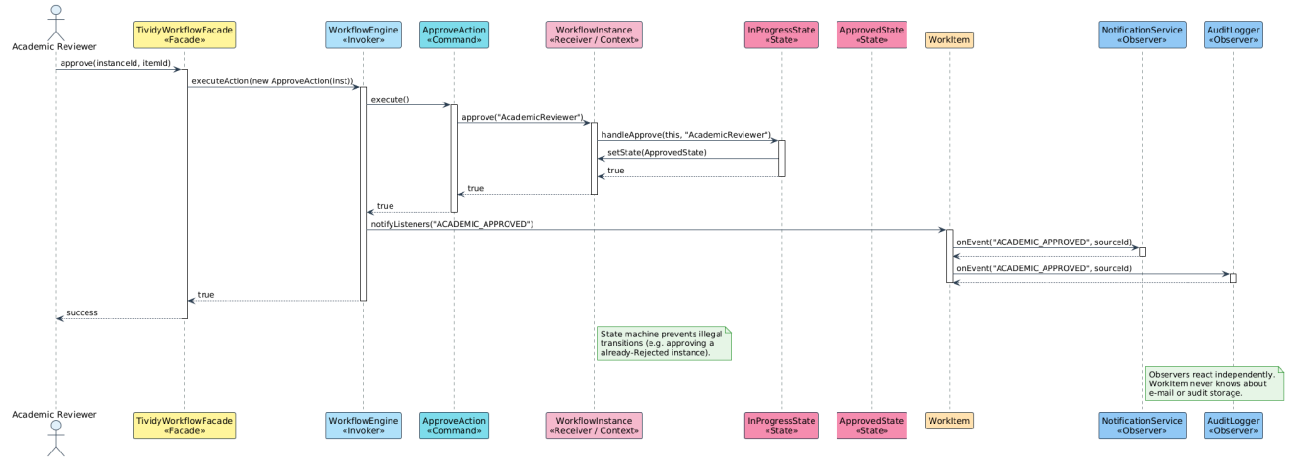
*updated UML Class diagram*

# Task 6: Runtime Behaviour Diagrams

## 6.1 UML Sequence Diagrams

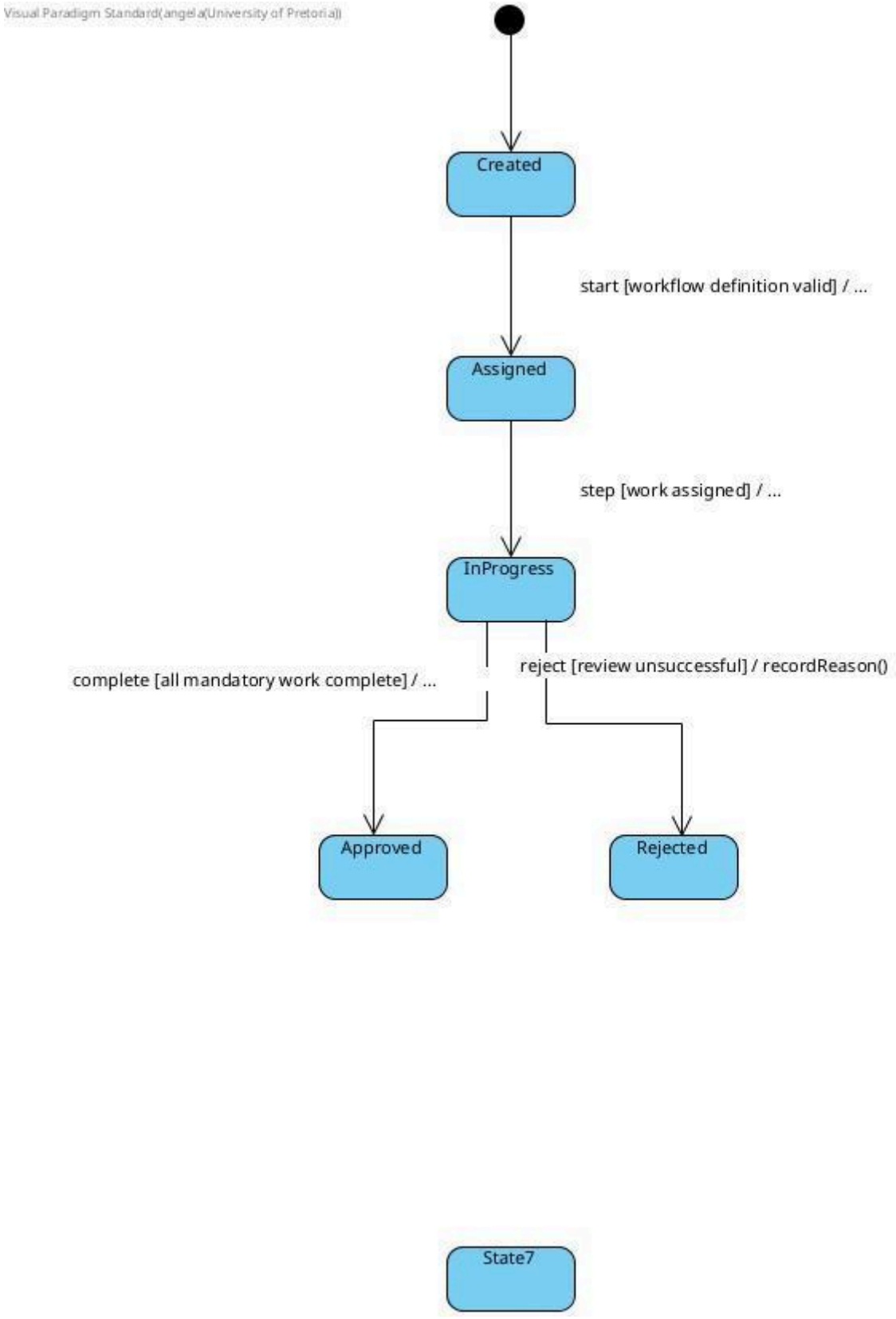



*Sequence Diagram 1: Creating and starting a workflow instance*




*Sequence Diagram 2: Completing a review task and notifying listeners*

## 6.2 UML State Diagram




*State Diagram: lifecycle of a running object*

## 7.1 Implementation Overview

TIVIDY was implemented in C++11 as an interactive, console-based workflow management system. The application separates its user interface, generic workflow engine and industry-specific workflow definitions to promote modularity and reusability.

The main application serves as the entry point, responsible for initialising the system and providing access to the interactive text-based user interface (TUI). Workflow execution is managed through the generic engine, which creates independent workflow instances, coordinates tasks, manages lifecycle transitions and records execution history.

Two industry-specific workflows, University Admissions and Bursary Processing and Logistics Shipment Exception Management, demonstrate how different business processes can be executed using the same core architecture.

The implementation incorporates Gang of Four design patterns to separate responsibilities such as workflow construction, task assignment, execution control, state management, coordination, event handling and recovery.

# Requirements Traceability

| Requirement | Implementing class(es) | Design pattern | Test evidence to reference |
|---|---|---|---|
| Create workflow definitions | **`WorkflowDirector`**, **`WorkflowDefinitionBuilder`** | Builder | Definition-construction test |
| Create independent instances | **`WorkflowEngine`**, **`WorkflowInstance`** | Builder, Composite | Instance-isolation test |
| Enforce lifecycle transitions | **`WorkItem`**, **`WorkItemState`** | State | Valid and invalid transition tests |
| Assign tasks to participants | **`Task`**, **`RoutingStrategy`** | Strategy | Routing test |
| Coordinate dependencies | **`DependencyMediator`** | Mediator | Dependency progression test |
| Handle approvals | **`ApprovalHandler`**, **`ApprovalRouting`** | Chain of Responsibility, Strategy | Approval/escalation test |
| Execute workflow actions | **`WorkflowEngine`**, **`WorkItemAction`** | Command | Action execution test |
| Save and restore state | **`WorkflowSnapshot`**, **`HistoryLog`** | Memento | Snapshot restoration test |
| Record workflow events | **`EventDispatcher`**, **`AuditLogger`** | Observer, Singleton | Event notification test |

# Testing

## GitHub Repository

https://github.com/patrickkaleo/COS214_P6

## Team Contribution Statement

**Katlego Sentsho u24876489**
Worked on activity diagram 2 and the state diagram. Implementing Facade and Command patterns

**Patrick Simuyemba u25632354**
Worked on sequence diagrams 1 and Github repository creation. Implemented ci/cd pipeline and builder design pattern for creating workflow definitions.

**Enock Nyamweya u25619871**
Worked on Activity Diagram 1 and came up with the branching strategy.

**Angelo Anthony u25573552**
Worked on Activity Diagram 3.

Worked on the mediator, chain of responsibility and iterator design patterns.

**Ntando Dlamini u25591267**
Worked on the class Diagram, Task 2, implemented the Memento design pattern and worked on the slides

**Angela Ramaboea u25445392**
Worked on task 1 and task 4 and overall compilation of the submission document.

Worked on the adapter , state design patterns, Worked on the tui.

**Tebogo Mokgwatsane**

Worked on sequence diagram 2 and implemented Composite and Decorator patterns